UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

Actividad Parcial 2

Problemas de búsqueda y espacios de búsqueda

Sección 2 · Temas 2.1 y 2.2 · Modelado + Código

Alumno(a):	SANVICENTE VALENCIA FERNANDO YAIR
Programa:	Ing. en Tecnologías de la Información e Innovación Digital
Asignatura:	Fundamentos de Inteligencia Artificial
Grupo:	171 · Cuatrimestre 7
Docente:	Mtro. Javier García Lara
Tipo de entrega:	Individual
Fecha de entrega:	18 de octubre de 2026

Archivos de la entrega: este PDF y el script `FIA_6171_Sanvicente_S2.py` (código completo en la sección 7).

1. Tema y sección

Sección: 2 de 6 (50 pts, individual). **Temas:** 2.1 Problemas de búsqueda y 2.2 Espacios de búsqueda. Se modelan dos problemas como problemas de búsqueda (estado, acciones, costo, meta), se representa cada espacio de estados como un grafo dirigido y ponderado, se programa en Python un generador de estados junto con el grafo en NetworkX, y se analiza la complejidad.

2. Datos del problema

Un problema de búsqueda se define con cinco elementos (Russell y Norvig, 2020): el **estado inicial**, las **acciones** aplicables en cada estado, el **modelo de transición** que indica a qué estado lleva cada acción, la **prueba de meta** y el **costo de camino**. El conjunto de estados alcanzables forma el espacio de estados, que se representa como un grafo: los nodos son estados y las aristas dirigidas son acciones con un peso igual a su costo.

Se eligieron dos problemas clásicos con espacios finitos y distintas estructuras de costo:

- **Problema 1: Jarras de agua.** Hay una jarra de 4 L y otra de 3 L, sin marcas de medida. Se debe lograr exactamente 2 L en la jarra de 4 L. Costo no uniforme: litros movidos en cada acción.
- **Problema 2: Misioneros y caníbales.** Tres misioneros y tres caníbales deben cruzar un río en un bote para dos personas. En ninguna orilla puede haber más caníbales que misioneros (si hay al menos un misionero). Costo uniforme: 1 por cruce.

3. Formulación

Formulación de ambos problemas como problemas de búsqueda:

Elemento	Problema 1: Jarras	Problema 2: Misioneros y caníbales
Estado	(a, b): litros en la jarra A (cap. 4) y en la B (cap. 3). Total posible: 5 × 4 = 20	(m, c, lado): misioneros y caníbales en la orilla izquierda y posición del bote (0 = izquierda, 1 = derecha)
Estado inicial	(0, 0)	(3, 3, 0)
Acciones	Llenar A, Llenar B, Vaciar A, Vaciar B, Verter A→B, Verter B→A (solo las aplicables)	Cruzar 1M, 2M, 1C, 2C o 1M+1C, en la dirección que corresponda al bote
Modelo de transición	Llenar: la jarra llega a su tope. Vaciar: queda en 0. Verter: se mueve min(contenido origen, espacio libre destino)	Se restan o suman los pasajeros según el lado del bote; solo se permite si el estado resultante es válido
Costo	Litros movidos en la acción (costo no uniforme)	1 por cruce (costo uniforme)
Prueba de meta	a = 2	(0, 0, 1): todos en la orilla derecha

4. Procedimiento

- **Modelado:** se definió el estado, las acciones y el costo de cada problema (tabla anterior).
- **Generador de estados:** para cada problema se programó una función generadora (`yield`) que, dado un estado, produce tuplas (acción, nuevo estado, costo).
- **Construcción del grafo:** se exploró todo el espacio alcanzable con un recorrido en anchura y cada transición se agregó como arista de un `DiGraph` de NetworkX, con los atributos `accion` y `weight`.

- **Resolución:** se obtuvo el camino de menor costo con el algoritmo de Dijkstra (equivalente a BFS cuando todos los costos son 1).
- **Visualización y análisis:** se dibujó cada grafo resaltando la solución y se midieron nodos, aristas y factor de ramificación para el análisis de complejidad.

5. Solución

5.1 Problema 1: Jarras de agua

El grafo resultante tiene 14 estados alcanzables (de 20 posibles) y 50 aristas dirigidas. El camino de menor costo tiene 6 acciones y un costo total de 16 litros movidos:

Paso	Acción	Estado resultante (a, b)	Costo
0	Estado inicial	(0, 0)	—
1	Llenar B	(0, 3)	3
2	Verter B → A	(3, 0)	3
3	Llenar B	(3, 3)	3
4	Verter B → A	(4, 2)	1
5	Vaciar A	(0, 2)	4
6	Verter B → A	(2, 0) — meta	2
	Total		**16**

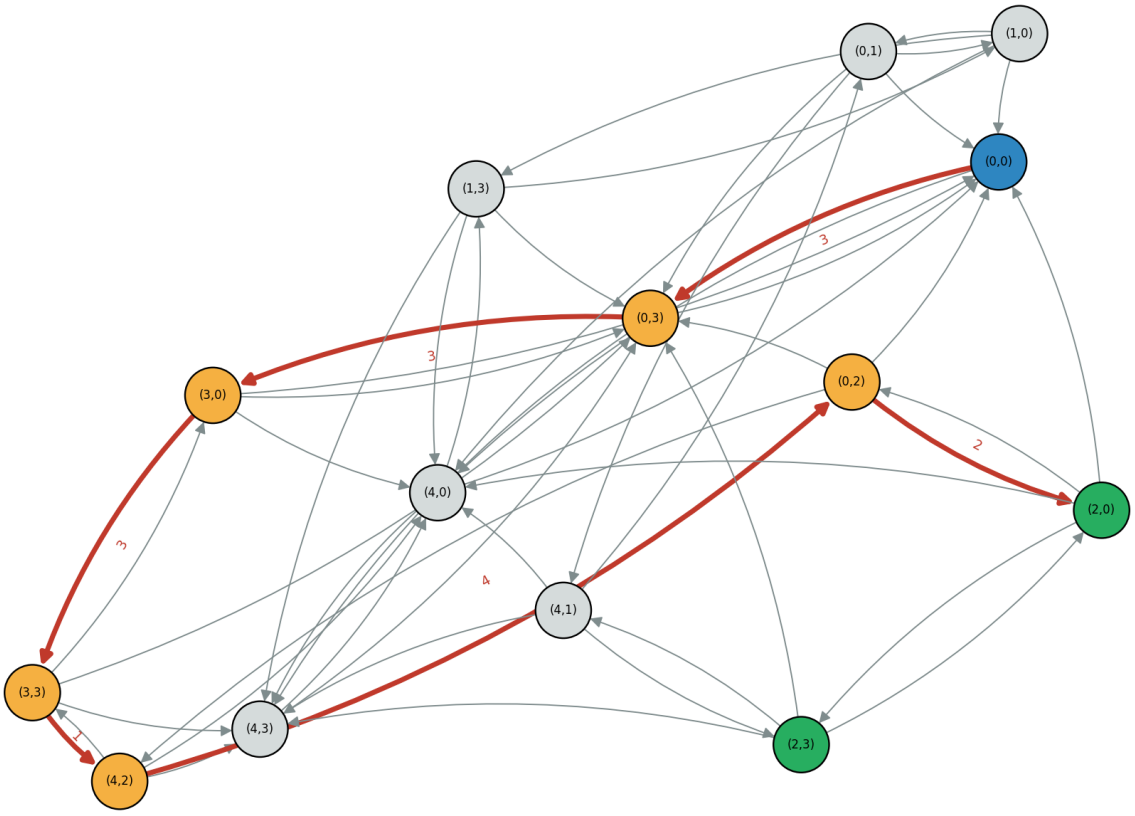


Figura 1. Grafo dirigido y ponderado del problema de las jarras. Azul: estado inicial; verde: estados meta; naranja y flechas rojas: solución óptima (con el peso de cada arista). Elaboración propia con NetworkX.

5.2 Problema 2: Misioneros y caníbales

De los 32 estados posibles solo 20 son válidos, y 16 son alcanzables desde el estado inicial. El grafo tiene 34 aristas dirigidas. La solución óptima requiere 11 cruces (costo 11), que coincide con la solución clásica conocida de este problema:

Paso	Acción	Estado (m, c, lado)	Paso	Acción	Estado (m, c, lado)
1	Cruzan 0M 2C	(3, 1, 1)	7	Cruzan 2M 0C	(0, 2, 1)
2	Regresa 0M 1C	(3, 2, 0)	8	Regresa 0M 1C	(0, 3, 0)
3	Cruzan 0M 2C	(3, 0, 1)	9	Cruzan 0M 2C	(0, 1, 1)
4	Regresa 0M 1C	(3, 1, 0)	10	Regresa 1M 0C	(1, 1, 0)
5	Cruzan 2M 0C	(1, 1, 1)	11	Cruzan 1M 1C	(0, 0, 1) — meta
6	Regresa 1M 1C	(2, 2, 0)			

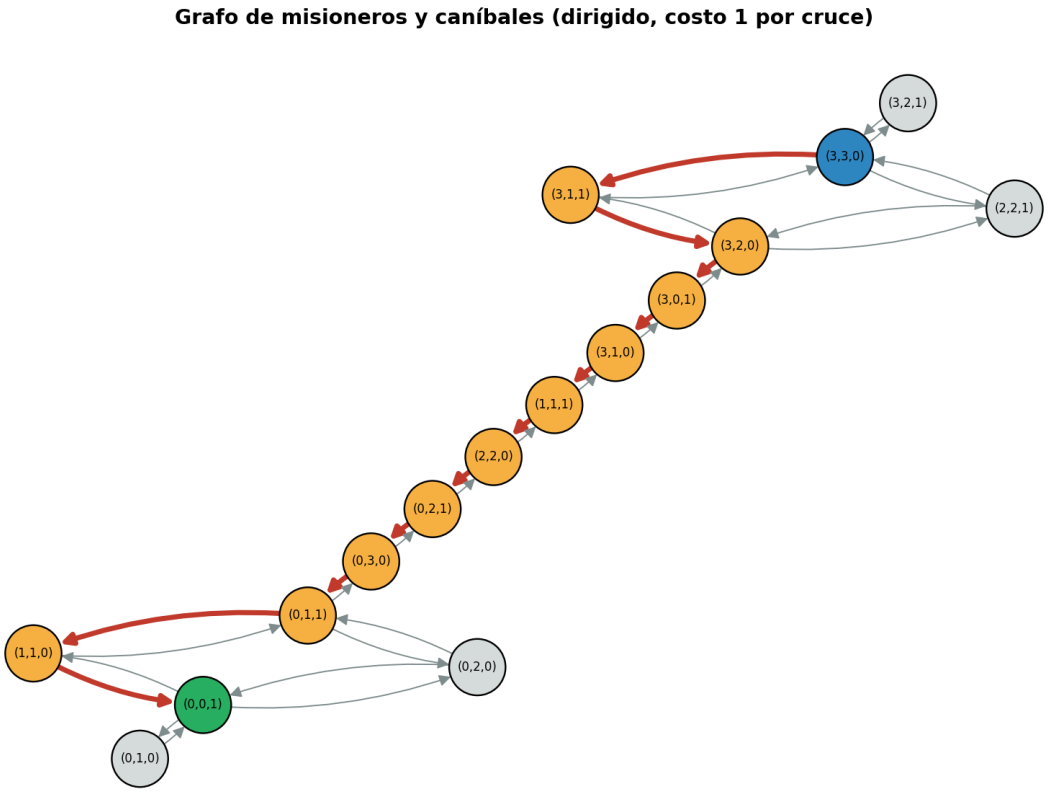


Figura 2. Grafo dirigido del problema de misioneros y caníbales (cada arista cuesta 1). Azul: estado inicial; verde: meta; naranja: camino óptimo. Elaboración propia con NetworkX.

6. Análisis de complejidad

Medida	Jarras	Misioneros y caníbales
Estados posibles	20	32 (4 × 4 × 2)
Estados válidos	20	20
Estados alcanzables |V|	14	16
Transiciones |E|	50	34
Ramificación promedio b (|E| / |V|)	3.57	2.12
Ramificación máxima	4	3
Profundidad de la solución d	6 acciones	11 cruces
Nodos de un árbol sin control de repetidos (b^d con b promedio)	≈ 2,000	≈ 3,900
Estados que explora una búsqueda en grafo	14	16

Complejidad en tiempo y espacio. En una búsqueda en árbol, que no recuerda los estados visitados, BFS crece como $O(b^d)$ en tiempo y espacio, y una búsqueda en profundidad (DFS) usa $O(b \cdot m)$ de memoria, con m la profundidad máxima. Como estos espacios tienen ciclos (por ejemplo, llenar y vaciar una jarra), un árbol repetiría estados una y otra vez. Al recordar los estados visitados, el recorrido pasa a $O(|V| + |E|)$ y Dijkstra con cola de prioridad a $O((|V| + |E|) \log |V|)$.

Lectura de los datos. En jarras, el árbol de búsqueda sin control de repetidos tendría del orden de miles de nodos para llegar a profundidad 6, mientras que el espacio real solo tiene 14 estados. En misioneros y caníbales la diferencia es todavía mayor: del orden de miles de nodos (con b promedio) frente a 16 estados reales, y hasta $3^{11} = 177{,}147$ con la ramificación máxima. La complejidad la determina el número de estados y no la profundidad de la solución, siempre que se detecten los repetidos.

Escalamiento. Con N misioneros y N caníbales el número de estados crece de forma polinómica, del orden de N^2, así que el problema sigue siendo manejable para valores grandes de N. En cambio, otros problemas como el 8-puzzle (181,440 estados alcanzables) o el 15-puzzle (del orden de 10^{13}) hacen inviable generar el grafo completo, y obligan a usar heurísticas, tema de las secciones siguientes.

7. Código en Python (comentado)

Dependencias: `networkx` y `matplotlib`. Ejecución: `python FIA_6171_Sanvicente_S2.py` (genera `grafo_jarras.png` y `grafo_misioneros.png` e imprime el reporte).

```
"""
Actividad Parcial 2 - Sección 2: Problemas de búsqueda y espacios de búsqueda
Alumno: Fernando Yair Sanvicente Valencia | Grupo 171
Asignatura: Fundamentos de Inteligencia Artificial (UP Atlautla)

Se modelan DOS problemas como problemas de búsqueda
(estado inicial, acciones, modelo de transición, costo, prueba de meta):
  1) Problema de las jarras (4 L y 3 L, meta: 2 L en la jarra grande)
     -> grafo dirigido y PONDERADO (costo = litros movidos)
  2) Misioneros y caníbales (3 y 3, bote de capacidad 2)
     -> grafo dirigido con costo uniforme (1 por cruce)
Cada espacio de estados se genera con un generador (yield) y se
representa con NetworkX (DiGraph).
"""
import networkx as nx
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# ============================================================================
# PROBLEMA 1: JARRAS DE AGUA
# Estado = (a, b): litros en la jarra A (cap. 4) y en la jarra B (cap. 3)
# ============================================================================
CAP_A, CAP_B = 4, 3
INICIO_J = (0, 0)

def meta_jarras(s):
    """Prueba de meta: la jarra A contiene exactamente 2 litros."""
    return s[0] == 2

def sucesores_jarras(s):
    """Generador de estados: produce (accion, nuevo_estado, costo).
    El costo es la cantidad de litros movidos (llenar/vaciar/verter)."""
    a, b = s
    # Llenar A o B hasta el tope (costo = litros agregados)
    if a < CAP_A: yield ("Llenar A", (CAP_A, b), CAP_A - a)
    if b < CAP_B: yield ("Llenar B", (a, CAP_B), CAP_B - b)
    # Vaciar A o B (costo = litros desechados)
    if a > 0: yield ("Vaciar A", (0, b), a)
    if b > 0: yield ("Vaciar B", (a, 0), b)
    # Verter A -> B
    t = min(a, CAP_B - b)
    if t > 0: yield ("Verter A->B", (a - t, b + t), t)
    # Verter B -> A
    t = min(b, CAP_A - a)
    if t > 0: yield ("Verter B->A", (a + t, b - t), t)

# ============================================================================
# PROBLEMA 2: MISIONEROS Y CANÍBALES
# Estado = (m, c, lado): misioneros y caníbales en la orilla IZQUIERDA;
# lado = 0 si el bote está a la izquierda, 1 si está a la derecha.
# ============================================================================
N = 3
INICIO_M = (N, N, 0)
META_M = (0, 0, 1)

def valido_mc(m, c):
    """Un estado es válido si en ninguna orilla hay más caníbales que
    misioneros (cuando hay al menos un misionero en esa orilla)."""
    if not (0 <= m <= N and 0 <= c <= N):
        return False
    mr, cr = N - m, N - c
    if m > 0 and c > m: return False
    if mr > 0 and cr > mr: return False
    return True

def sucesores_mc(s):
    """Generador de estados: el bote lleva 1 o 2 personas; costo = 1."""
    m, c, lado = s
    for dm, dc in [(1,0),(2,0),(0,1),(0,2),(1,1)]:
        if lado == 0:   # el bote sale de la izquierda
            nm, nc = m - dm, c - dc
        else:           # el bote regresa de la derecha
            nm, nc = m + dm, c + dc
        if valido_mc(nm, nc):
```

```
            yield (f"Cruzan {dm}M {dc}C", (nm, nc, 1 - lado), 1)

# ======================================================================
# Construcción del grafo (exploración completa del espacio de estados)
# ======================================================================
def construir_grafo(inicio, sucesores):
    G = nx.DiGraph()
    G.add_node(inicio)
    visitados, frontera = {inicio}, [inicio]
    while frontera:
        s = frontera.pop(0)
        for accion, nuevo, costo in sucesores(s):
            G.add_edge(s, nuevo, accion=accion, weight=costo)
            if nuevo not in visitados:
                visitados.add(nuevo)
                frontera.append(nuevo)
    return G

def resolver(G, inicio, es_meta):
    """Camino de menor costo con Dijkstra (BFS si todos los costos son 1)."""
    metas = [n for n in G.nodes if es_meta(n)]
    mejor = min(((nx.dijkstra_path_length(G, inicio, m), m) for m in metas
                 if nx.has_path(G, inicio, m)), default=None)
    if mejor is None:
        return None, None
    costo, meta = mejor
    return nx.dijkstra_path(G, inicio, meta), costo

def dibujar(G, camino, inicio, es_meta, titulo, archivo, etiquetas_peso=True, size=(11, 8), layout="kk"):
    pos = nx.kamada_kawai_layout(G) if layout == "kk" else nx.spring_layout(G, seed=7, k=1.2)
    en_camino = set(zip(camino, camino[1:])) if camino else set()
    colores = []
    for n in G.nodes:
        colores.append("#2E86C1" if n == inicio else "#27AE60" if es_meta(n) else "#F5B041" if n in camino else "#D5DBDB")
    ancho = [3.0 if e in en_camino else 0.8 for e in G.edges]
    ecol = ["#C0392B" if e in en_camino else "#7F8C8D" for e in G.edges]
    plt.figure(figsize=size)
    nx.draw_networkx_nodes(G, pos, node_color=colores, node_size=1100, edgecolors="black")
    nx.draw_networkx_labels(G, pos, labels={n: str(n).replace(" ", "") for n in G.nodes}, font_size=7)
    nx.draw_networkx_edges(G, pos, width=ancho, edge_color=ecol, arrows=True, arrowsize=14,
                           connectionstyle="arc3,rad=0.12", node_size=1100)
    if etiquetas_peso:
        pesos = {e: G.edges[e]["weight"] for e in G.edges if e in en_camino}
        nx.draw_networkx_edge_labels(G, pos, edge_labels=pesos, font_size=8, font_color="#C0392B",
                                     label_pos=0.5, bbox=dict(alpha=0.0))
    plt.title(titulo, fontsize=12, fontweight="bold")
    plt.axis("off"); plt.tight_layout()
    plt.savefig(archivo, dpi=170); plt.close()

def reporte(nombre, G, inicio, es_meta, camino, costo):
    print("=" * 62); print(nombre); print("=" * 62)
    print(f"Estados alcanzables |V| = {G.number_of_nodes()}")
    print(f"Transiciones         |E| = {G.number_of_edges()}")
    print(f"Factor de ramificación promedio b = {G.number_of_edges()/G.number_of_nodes():.2f}")
    print(f"Factor de ramificación máximo     = {max(d for _, d in G.out_degree())}")
    if camino:
        print(f"Solución óptima: {len(camino)-1} acciones, costo total = {costo}")
        for u, v in zip(camino, camino[1:]):
            print(f"   {u} --[{G.edges[u, v]['accion']} | costo {G.edges[u, v]['weight']}]--> {v}")
    print()

if __name__ == "__main__":
    # ---- Problema 1 ----
    G1 = construir_grafo(INICIO_J, sucesores_jarras)
    c1, k1 = resolver(G1, INICIO_J, meta_jarras)
    reporte("PROBLEMA 1: JARRAS (4 L y 3 L, meta 2 L en A)", G1, INICIO_J, meta_jarras, c1, k1)
    dibujar(G1, c1, INICIO_J, meta_jarras, "Grafo del problema de las jarras (dirigido y ponderado)", "grafo_jarras.png")

    # ---- Problema 2 ----
    G2 = construir_grafo(INICIO_M, sucesores_mc)
    c2, k2 = resolver(G2, INICIO_M, lambda s: s == META_M)
    reporte("PROBLEMA 2: MISIONEROS Y CANIBALES (3 y 3)", G2, INICIO_M, lambda s: s == META_M, c2, k2)
    dibujar(G2, c2, INICIO_M, lambda s: s == META_M, "Grafo de misioneros y caníbales (dirigido, costo 1 por cruce)",
            "grafo_misioneros.png", etiquetas_peso=False, size=(10, 7))

    # Cantidad total de estados posibles vs. alcanzables (para el análisis)
    print("Jarras: estados posibles (CAP_A+1)*(CAP_B+1) =", (CAP_A + 1) * (CAP_B + 1))
    posibles_mc = sum(1 for m in range(N+1) for c in range(N+1) for l in (0, 1) if valido_mc(m, c))
    print("M&C: estados válidos (incluyendo ambos lados del bote) =", posibles_mc)
```

8. Conclusiones

- Modelar un problema como búsqueda exige definir con precisión el estado, las acciones, el costo y la meta; una vez hecho, la solución se reduce a encontrar un camino en un grafo.
- El generador de estados separa las *reglas del problema* del *algoritmo de búsqueda*: el mismo código de construcción del grafo y de Dijkstra sirvió para los dos problemas, solo cambió la función sucesora.
- Con costos no uniformes (jarras) hace falta un algoritmo que los tome en cuenta, como Dijkstra; con costo uniforme (misioneros y caníbales) BFS da el mismo resultado.
- Recordar los estados visitados cambia por completo la complejidad: de miles de nodos en un árbol a 14 y 16 estados reales. Para espacios mucho mayores esto ya no basta y se necesitan heurísticas o búsqueda local.
- La solución obtenida para misioneros y caníbales (11 cruces) coincide con la solución clásica del problema, lo que sirve como verificación del modelo.

Referencias (APA 7.ª ed.)

Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. En G. Varoquaux, T. Vaught, & J. Millman (Eds.), *Proceedings of the 7th Python in Science Conference* (pp. 11-15).

NetworkX Developers. (s. f.). *NetworkX documentation*. https://networkx.org/documentation/stable/

Russell, S., & Norvig, P. (2020). *Artificial intelligence: A modern approach* (4.ª ed.). Pearson.

Declaración de uso de IA.
Herramienta: Claude (Anthropic).
Prompt: «Empieza la Sección 2: modelar 2 problemas de búsqueda, representarlos como grafo dirigido y ponderado, programar en Python un generador de estados y el grafo con NetworkX, con diagrama, código comentado y análisis de complejidad».
Uso: propuesta de los problemas, escritura del código, diagramas y borrador del análisis.
Verificación: el código se ejecutó y sus resultados (número de estados, aristas y camino óptimo) se tomaron de esa ejecución; la solución de 11 cruces de misioneros y caníbales se comparó con la solución clásica conocida. El contenido fue revisado y ajustado por el alumno (Fernando Yair Sanvicente Valencia).